Disjoint Sets. → used in dynamic graphs
↳ Used to find Parent
↳ Union by rank.

→ Time complexity → O(4α)
↳ O(1)

→ (1,2) ✓
(2,3) ✓
(4,5) ✓
(6,7) ✓
(5,6) ✓
(3,7)

rank →

1	2	3	4	5	6	7
0	0	0	0	0	0	0

parent →

1	2	3	4	5	6	7
1	1	1	4	4	6	7

initially every one is a parent of themselves.

Union (u,v)

i) Find ultimate parents of u & v, pu & pv.

ii) Find rank of ultimate parents

iii) Never connect smaller rank to large, rank always.

initial counting

(1,2) pu=1, pv=1, r=0, r=0

(2,3) pu=1, pv=3, r=1, r=0

rank will not change as height is still 1

(4,5) pu=4, pv=5, r=0, r=0

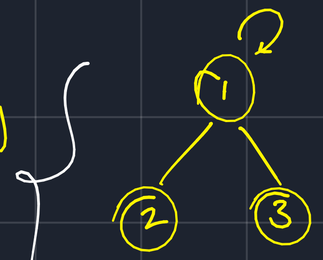
pu=6, pv=7, r=0, r=0

(5,6) pv=4, pv=6, r=1, r=1

→ finding parent as of now will take $\log(n)$ time.

↳ Path compression is applied to make it constant.

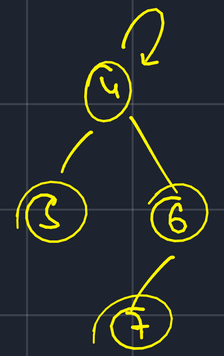
→ currently



↳ takes $\log(n)$ space

$pu = 7$

$7 \to 6 \to 4$



(3,7)

$pu = 1$ $pu = 4$

$r = 1$ $r = 2$

→ Does 1 & 7 belong to the same component.

↳ check the ultimate parents of both

↳ same → yes

↳ ≠ → no

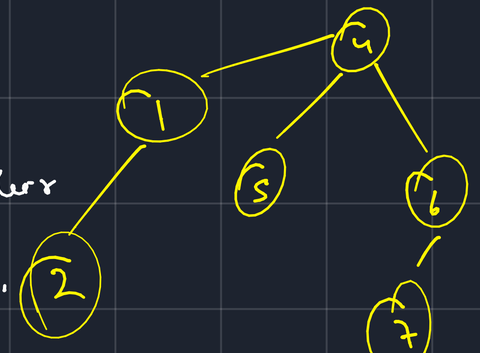
↳ Path compression.

↳ this skews the tree up.

↳ Ideally, the rank should be reduced

→ this link is removed

↳ all nodes store their ultimate parent.

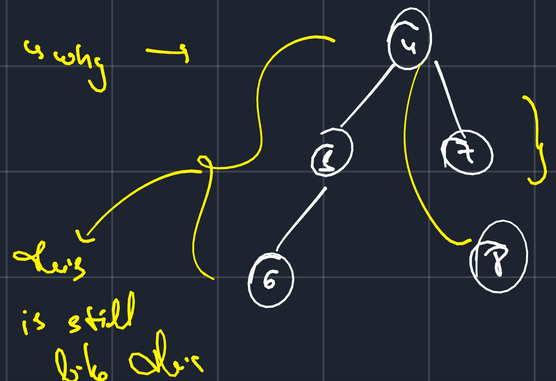


$4 \to 3 \to 2 \to 1$

$p[4] = 1$ } constant

$p[2] = 1$

↳ Time complexity of union → $O(4\alpha)$



↳ why →

this is still b/c this

p(8)

→ acc. to this we should decrease the rank

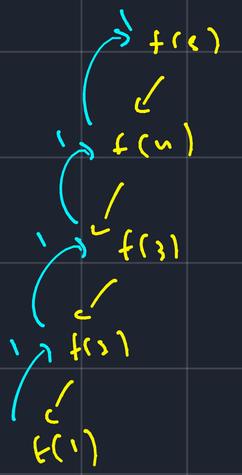
but we cannot

↳ Name to path compress.

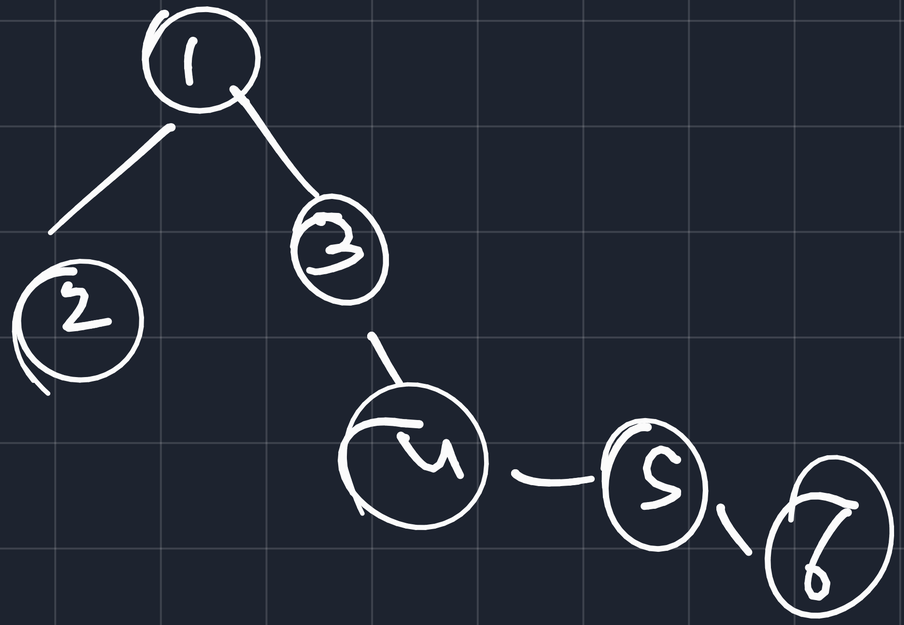
```
fn P(u)
{
    if (u == par(u))
        ret u;

return par(u) = find · P(par(u))
}
```

Path compression.



→ why connect smaller to larger?



Is connecting larger to smaller will make us traverse larger distance.

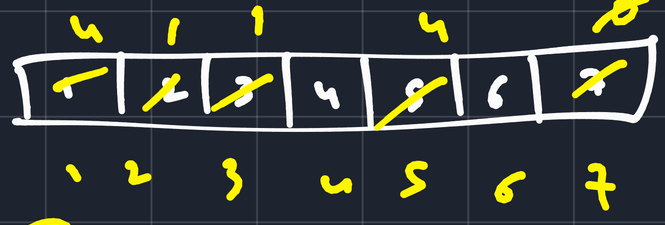
& travel times for all nodes will be larger

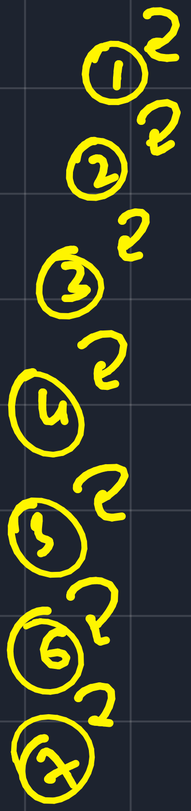
→ union by size.

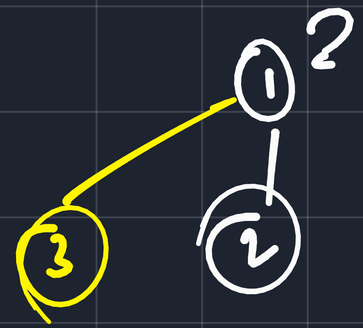
↳ size →

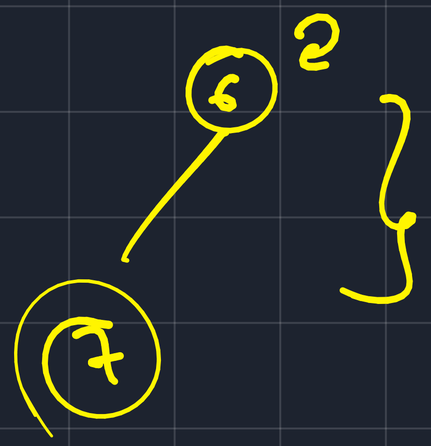
keeping a track how big is the tree

↳ parent →

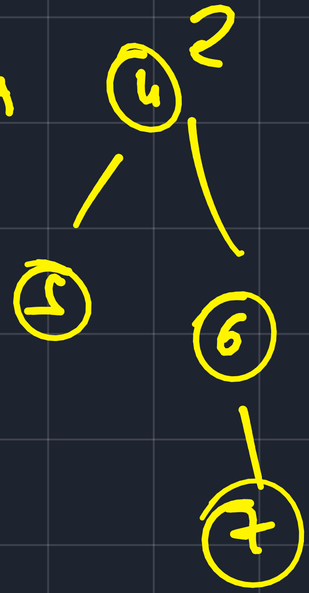








→



↳